Research of Industrial Control System Device Firmware Vulnerability Mining Technology Based on Taint Analysis

Yi Li, Xiaoman Liu, Huirong Tian and Cheng Luo
Security Research Institute
China Academy of Information and Communication Technology (CAICT)
No. 52 Huayuanbei Road, Haidian District, Beijing 100191, China
{liyi, liuxiaoman, tianhuirong & luocheng}@caict.ac.cn

Abstract—**Aiming** at the problem that there is little research on firmware vulnerability mining and the traditional method of vulnerability mining based on fuzzing test is inefficient, this paper proposed a new method of mining vulnerabilities in industrial control **system** firmware. Based on taint analysis technology, this method can construct test cases specifically for the variables that may trigger vulnerabilities, thus reducing the number of invalid test cases and lmprovlng the test efficiency. Expertment result shows that this method can reduce about 23% of test cases and can effectively improve test efficiency.

Keywords-component; industrial control system; device firmware; vulnerability mining; taint analysis

I. INTRODUCTION

With the development of informatization in industry, there is more and more traditional network technology applied in industrial control systems. On one hand, this improves industrial production, while on the other hand, it has brought in many security problems lie outside ICSs before. Although industrial enterprises have generally improved the degree of attention to ICS security protection after the incident of Stuxnet and the attack against the power system of Ukraine, but in recent years, security incidents related to ICSs still emerge endlessly. A market research consultancy named Business Advantage conducts an interview together with Kaspersky Lab to 359 companies from 21 countries last year, the result shows that 54% of them have had at least one security incident in their ICSs in one year. And among security threats these companies faced, malware and APT are ranked the top [1]. Most of these threats make use of vulnerabilities in ICS software or hardware to achieve their attack purpose. Therefore, to discover and repair of vulnerabilities in ICSs in time become one of the important means of ICS security protection. And since 2016, an important security trend is that the security problems exposed by terminal devices are increasing, and it has become an important driving force for attackers. Cases like controlling large amounts of terminal devices to perform cryptocurrency mining or DDoS attack on a targeted system have been exposed [2]. Thus how to explore the possible vulnerabilities in ICSs to cut off the potential attack path of attackers, and in tum to prevent the industrial control devices from becoming the tool of attackers has become a research hotspot.

At present, research about ICS vulnerability mining mainly focuses on the field of industrial control network protocol and control software both at home and abroad, and the attention paid to firmware vulnerability mining is relatively little, and the existing mining methods usually build test cases based on random variation of input data, the efficiency of mining is quite low [3]-[7]. Aimed at this problem, this paper studies the vulnerability mining technology of industrial control devices, and proposes a method of vulnerability mining for industrial control device firmware. Based on the taint analysis technology, this method can identify the focus of the test before the test is carried out and build test cases accordingly, thus reducing the number of invalid test cases and improving the test efficiency. Experimental result shows that this method only needs 76.6% of test cases to find the same number of vulnerabilities compared to the traditional method, thus effectively improves the test efficiency.

The other parts of this paper are organized as follows. Part 2 introduces the principle of taint analysis. Part 3 introduces the firmware vulnerability mining model, taint tracking algorithm and test case generation method based on taint analysis. Part 4 makes a comparative analysis of the traditional method of vulnerability mining and the improved method of vulnerability mining. Part 5 summarizes the work of this paper.

II. BRIEF INTRODUCTION OF TAINT ANALYSIS

Taint analysis (also known as information flow tracking technology) is a practical method of information flow analysis technology, which detects system security problems by marking sensitive data in the system and then tracking the spread of marked data in the process [8].

Taint analysis can be abstracted as a triple *(sources, sinks, sanitizersi*, among them, *sources* is the source of taints, it means that untrusted or confidential data is instructed into the system. *sinks* are sink points, which represents the direct generation of security sensitive operations (which violates data integrity) or the leakage of private data to the outside (which violates data confidentiality). *saniizers* are

This work is supported by National Key R&D Program of China (program No. 2016YFB0800202).

978-1-5386-6565-7/18/$31.00 ©2018 IEEE

harmless processes, which means data transmission is no longer harmful to the infonnation security of the software system by means of data encryption or removal of hazards. Taint analysis is to analyze whether the data introduced by the taint source in the program can be transmitted directly to the sink points without harmless processing. If not, the system is secure in the aspect of infonnation flow; otherwise, it indicates that the system has security problems such as privacy data leakage or dangerous data operation. The process of taint analysis is shown in Figure 1 [9].

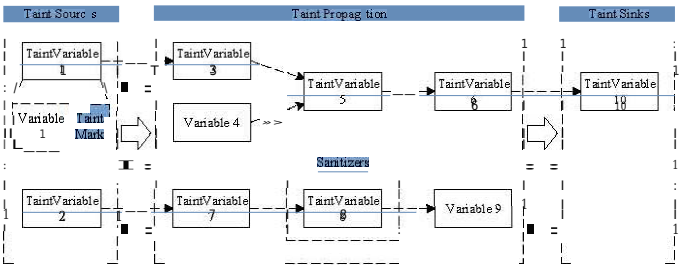

Figure 1. Process of taint analysis

In the practice of solving application security problems, taint analysis needs the support of program analysis technology. For example, static taint analysis technology can determine all possible propagation paths of taint marks by analyzing static dependencies between statements or instructions in source code or bytecode, Dynamic taint analysis technology can be used to track the spread of taint marks through program piling combined with customized hardware. Thus taint analysis does not have to change the original programming model or language features of the application, but provides precise tracking of data flow propagation

III. METIIOD OF VULNERABILITY MINING

A. *Mathematical model a/taint analysis in vulnerability mining*

Before introducing the method of vulnerability mining, a group of definitions are given:

Definition 1 define the program segment under detection as p, and define the function $EOP(p) = \{true, [alse\}$. If $EOP(p) = true$, it means that the program segment detection is finished. If $EOP(p) = false$, it means that the program segment detection is not finished.

Definition 2 define the input variable set of the program segment under detection as L, and define the taint source set as I, thus $I \subseteq L$.

Definition 3 define the instruction set of the program segment under detection as C, and define sensitive instruction set as H, thus H $\subseteq$ C. Sensitive instructions mainly include the following categories:

① functions that handles strings, such as string copy functions.

② functions that execute commands, such as file operation functions, user permission promotion functions, etc.

@ authentication operations, such as user login operations.

④ important resource operations, such as database and configuration content operations.

Definition 4 define M as the set of variables in memory of the program segment under detection, and define R as the set of variables in registers of the program segment under detection.

Definition 5 define V as the set of all variables, thus V is the union of M and R, in other words, $V = \{v \mid V \in M \cup R\}$

Definition 6 define M_T to represent the address set of the tainted memory region and define R_T to represent the address set of the tainted registers. Thus $M_T \subseteq M$, $R_T \subseteq R$.

Definition 7 define V_T as the set of tainted variables, thus V_T is the union of M_T and R_T, that is to say, $V_T = \{v_T | v_T \in M_T \cup R_T\} \subseteq V$. For $\forall v \in V$, if a variable v was written into data in I, M_T or R_T, the variable v is called tainted.

Definition 8 define function $S(v)$ which can return the taint source set of variable v, thus $S(v) \subseteq I$.

Definition 9 define function $D(v)$ which can return the set of sensitive instructions that operate variable v, thus $D(v) \subseteq$ H. When there is no sensitive instruction exist in instructions operating variable v, $D(v) = \phi$.

Definition 10 the state of each variable in the program segment under detection can be represented by a state machine in the fonn ofFonnula 1:

$$M = (Q, I, T, (J, Q_o) \quad (1)$$

Among them, Q is the set of states a variable can have during the process of taint analysis, $Q = \{Q_n, O; Q_s' Q_i\}$, where Q_n represents the normal state and the variable is not tainted at this time. Q_t is the tainted state, at which time the variable is tainted. Q_s is the safe state and at this time, the variable will neither leak sensitive information through the interface function nor cause the program to run abnormal because the variable takes abnormal value. Q_i is the unsafe state and at this time, the variable may leak sensitive information through the interface function or cause the program to run abnormal because the variable takes an abnormal value. Meanwhile, define $Q_s > Q_n > Q_t > Q_i$. Σ is the set of input events, $\Sigma = \{e \mid e \in S(v) \times D(v) \times EOP(p)\}$ and in any determined moment, the state machine M can receive only a certain input. T is the set of state transitions. σ is the mapping function, $(J = Q \times \Sigma \rightarrow T$. Q_0 is the initial state, $Q_o = Q_n'$ The state transition relationship of state machine M is shown in Figure 2.

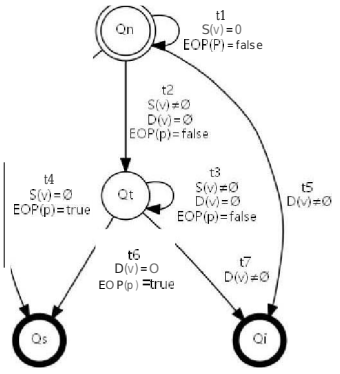

Figure 2. State transition diagram of each variable in the program segment under detection in vulnerability mining state machine

Definition 11 define function $F(v)$ as the final state of a variable after the program segment detection finished. Thus $F(v)$ E $\{Qs' Q_i\}$. For illustration purposes, define $F_i(v)$ as the state that the variable V returns after instruction i is executed.

Definition 12 if $F(v) = Qs$, then $S(v) = \phi$.

Definition 13 for variable $v = x + y$, define operation $F(v) = min\{F(x), F(y)\}$. It means that when a variable combines another one and becomes a combination variable, then $F(v)$ returns the lower bound of the function value returned by 2 variables respectively. This definition indicates that if a variable is composed of tainted variables and untainted variables, then the composed variable should be tainted after execution. The reason is that the research of taint analysis is actually to find the spreading process of the taint, and the changes of the state can only go from the untainted state to the tainted state, while cannot go in the opposite direction. If not, the process of vulnerability mining will lose its significance. Note that the initial state does not participate in the operation.

B. *Taint tracking algorithm*

The taint tracking algorithm is a loop process. Each instruction of the detected program segment is analyzed to determine whether it is involved in the spread of taint variables and whether it is a sensitive instruction. Detailed steps are explained as follows:

Step 1: establish the state machine as shown in Figure 2 for each variable in the program segment under detection, and set the initial state as normal state.

Step 2: start from the first instruction of the program segment under detection, analyze the instruction in the current running position, take the instruction, judge its type, instruction function, instruction source operand, and instruction destination operand, and prepare for the next judgment. Then go to Step 3.

Step 3: judge whether this instruction has data writing function or is a jump instruction according to the instruction function and type obtained by Step 2. If neither is, go forward one step and return to Step 2. If this instruction has data writing function and not a jump instruction, go to Step 4. Otherwise, go to Step 5.

Step 4: take the operand, andjudge whether the destination operand exists the propagation of tainted data. That is, judge whether the destination operand is related to the tainted source according to the definition of the aforementioned state model. If so, set the variable to the tainted state, and then go to Step 6. Otherwise, step forward and return to Step 3.

Step 5: if the instruction is ajump instruction, judge whether the jump parameter is related to the tainted data. If the parameter is derived from the taint variable, go forward one step and then go to Step 2. Otherwise, jump to the target instruction and go to Step 2.

Step 6: judge the instruction in Step 4 according to the above definition of sensitive instruction. If so, set the variable associated with the operand into unsafe state. At the same time, find the taint source and increment the weight of the associated field by 1. If this instruction is the last instruction of the program segment being detected, go to Step 7. Otherwise, step forward and go to Step 2.

Step 7: set all variables in normal state and tainted state to safe state.

The process of taint tracking algorithm is shown in Figure 3.

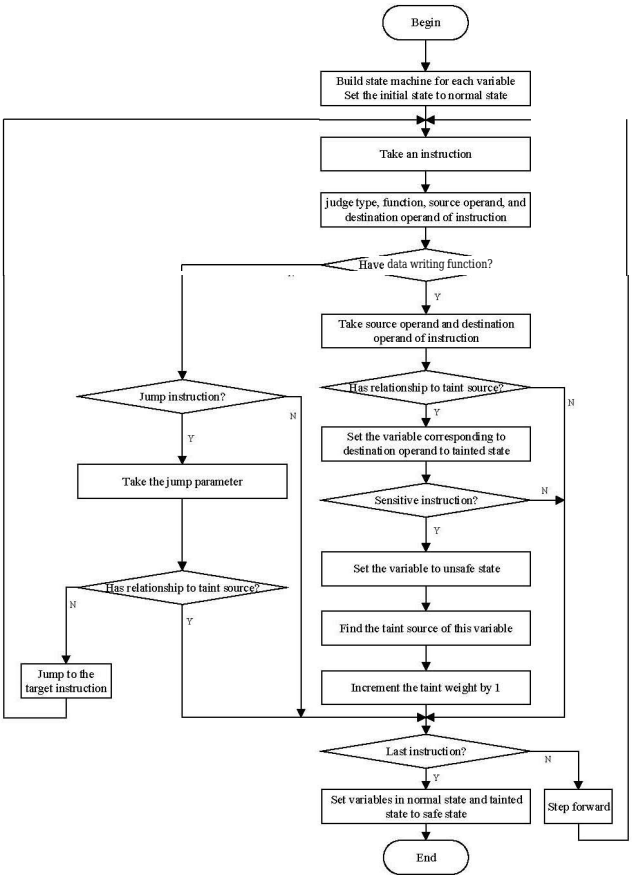

Figure 3. Flow of taint tracking algorithm

C. *Test case generation method*

After using the taint tracking algorithm to obtain a set of variables in unsafe state for the program segment being detected,

test cases need to be built against those variables. According to the work practice of vulnerability mining, the greater the sensitivity weight means that the variable is more likely to leak sensitive information through the interface function or cause the program to run abnormal due to the abnormal value of the variable, which means the probability of discovering the vulnerability is higher. A higher probability means that more test cases are needed to trigger a vulnerability, so in this article, the number of test cases for a particular variable is proportional to its sensitive weight.

Define the set of variables in unsafe state as $U = \{Vi!' Vi2, \ldots, Vi_n\}'$ define its corresponding sensitive weight set as $W = \{w_1, wZ, \ldots, W_n\}$, and define the total amount of test cases as N, thus for $\forall v_{i_k}$ E U, the number of test cases for Vi_k can be depicted as $N_{Vk} = N.w_k/\sum_{j=1}^{n} w_j$.

IV. EXPERIMENT AND ANALYSIS

In order to verify the effectiveness of the method proposed in this paper, a prototype system is constructed to analyse the effect of vulnerability mining using several PLC firmware currently used in the market. The operating systems of the experimental environment are Windows 10 64-bit, Kali Linux 32-bit and Windows XP 32-bit. Among them, Kali and Windows XP run in VirtualBox 5.2.0. The 3rd party software and tools include Metasploit, Binwalk, Firmware Mod Kit, IDA pro 6.3, Sulley, etc [10]. Due to high requirements on hardware for tools and virtual machines, hardware configuration includes Intel core is 6200U 2.4GHz and 12GB memory. All of these software or tools can run well on this hardware platform.

Through testing several PLC firmware, it is found that the number of test cases required by the method proposed in this paper is about 76.6% of the number oftest cases required by the traditional method when the same number of vulnerabilities are triggered. It is shown that the method proposed in this paper has higher mining efficiency than the traditional method. The experimental result is shown in Table 1.

TABLE I. COMPARISON OF THE NUMBER OF TEST CASES REQUIRED FOR DIFFERENTPLC FIRMWARE MINING UNDER THE SAME NUMBER OF VULNERABILITIES

No. of PLC Firmware	Amout of Vulnerability	Amount of Test Cases Needed	
		Traditional method	***Method proposed by this paper***
1	3	356	279
2	5	674	518
3	6	903	674

V. CONCLUSION

As a new direction of research in recent years, industrial control system security has been widely concerned. Security incidents ofICS occur from time to time, and the number of open vulnerabilities keeps rising. But recently industrial control system vulnerabilities mining research work mainly focus on the control protocol and control software, and there is less attention paid for industrial control device firmware vulnerability mining, and most of existing methods are implemented by constructing test cases randomly, which has a poorer mining efficiency, and thus the industry urgently needs a kind of mining method for industrial control device firmware. In this paper, a method of vulnerability mining for industrial control device firmware is proposed. This method uses the taint analysis method to filter the variables that may be vulnerable in the program segment being detected, and builds test cases against these variables to reduce the number of invalid test cases and improve the test efficiency. Experimental result shows that this method can save an average of about a quarter of the number of test cases and can significantly improve the efficiency of vulnerability mining when the same number of vulnerabilities are mined. In the future work, the method of generating vulnerability test cases can be further optimized to improve the efficiency of vulnerability mmmg.

ACKNOWLEDGMENT

This work is supported by National Key R&D Program of China (program No. 2016YFB0800202). The author of this paper will also appreciate for the hard work of peer reviewers and the support of China Academy of Information and Communication Technology (CAlCT).

REFERENCES

[1] Business Advantage and Kaspersky Lab. The State of Industrial Cybersecurity 2017.2017.

[2] Kaspersky Lab. Threat Landscape for Industrial Automation Systems HI 2018.2018.

[3] Rashid A, Rashid A, Rashid A, et al. SENAMI: Selective Non-Invasive Active Monitoring for ICS Intrusion Detection. ACM Workshop on Cyber-Physical Systems Security and Privacy. ACM, 2016, pp. 23-34.

[4] ZHANG Xiao-ming, WANG Li-hong, HE Yue-ying, HE Shi-ping. Analysis of potential vulnerabilities and security testing in industrial control system. Chinese Journal on Internet of Things, 2017, vol. 1(1), pp. 34-39.

[5] LI H, DONG W, ZHU G Y. Research on industrial control protocol vulnerability mining technology based on Fuzzing test. Application of Electronic Technique, 2016, vol. 42(7), pp. 79-82.

[6] WANG Kai. Research on the Security Detection Technology ofa Typical Industrial Control System. Beijing University of Posts and Telecommunications, 2016.

[7] YU Changqi. The Study of Industrial Control System Device Vulnerability Discovery. Beijing University of Posts and Telecommunications, 2015.

[8] WANG Lei, LI Feng, LI Lian, FENG Xiao-Bing. Principle and Practice of Taint Analysis. Journal of Software, 2017, vol. 28(4), pp. 860-882.

[9] DAI Zhonghua, ZHAO Bol, WANG Ting, ZOU Li. A Fuzzing Test Method for Embedded Device Firmware Based on Taint Analysis. Journal of Sichuan University (Engineering Science Edition). 2016, vol. 48(2), pp. 125-131.

[10] Liu J, Guangyuan F U, Hailong L I. Research on VxWorks System Vulnerability Mining and Security Assessment. 2nd International Conference on Artificial Intelligence and Engineering Applications (AIEA 2017),2017, pp. 634-643.